Radarr Title Search Logic Walkthrough

Overview

Radarr has been successfully cloned to `/home/alex/.gemini/antigravity/scratch/radarr`. This walkthrough explains how Radarr searches for movie releases using alternative titles, translations, and various title variations.

Key Components

1. Alternative Title Model

File: AlternativeTitle.cs

```
public class AlternativeTitle : Entity<AlternativeTitle>
{
    public SourceType SourceType { get; set; }
    public int MovieMetadataId { get; set; }
    public string Title { get; set; }
    public string CleanTitle { get; set; }  // Normalized for searching
}

public enum SourceType
{
    Tmdb = 0,         // From TMDb metadata
    Mappings = 1,     // From external mappings
    User = 2,         // User-defined
    Indexer = 3       // From indexer responses
}
```

Key Points: - Alternative titles are stored with both original and "clean" versions - Clean titles are normalized using `CleanMovieTitle()` for consistent matching - Multiple sources can provide alternative titles

---

2. Alternative Title Management

File: AlternativeTitleService.cs

The service manages alternative titles with intelligent deduplication:

```
public List<AlternativeTitle> UpdateTitles(List<AlternativeTitle> titles, MovieMetadata movi
{
    // 1. Remove titles matching the main title
    titles = titles.Where(t => t.CleanTitle != movieMetadata.CleanTitle).ToList();

    // 2. Remove duplicate clean titles
    titles = titles.DistinctBy(t => t.CleanTitle).ToList();
```

```
    // 3. Prevent conflicts with other movies' titles
    var allTitlesByCleanTitles = _titleRepo.FindByCleanTitles(titles.Select(t => t.CleanTitl
    titles = titles.Where(t => !allTitlesByCleanTitles.Any(e =>
        e.CleanTitle == t.CleanTitle && e.MovieMetadataId != t.MovieMetadataId)).ToList();

    // 4. Update/insert/delete as needed
}
```

Protection Mechanisms: - Prevents alternative titles from conflicting with main titles - Ensures uniqueness within a movie - Prevents cross-movie title conflicts

---

3. Search Execution Flow

File: ReleaseSearchService.cs

Search Criteria Building When searching for a movie, Radarr builds a comprehensive list of titles to search:

```
private TSpec Get<TSpec>(Movie movie, bool userInvokedSearch, bool interactiveSearch)
{
    var wantedLanguages = _qualityProfileService.GetAcceptableLanguages(movie.QualityProfile
    var translations = _movieTranslationService.GetAllTranslationsForMovieMetadata(movie.Mov

    var queryTranslations = new List<string>
    {
        movie.MovieMetadata.Value.Title,           // Main title
        movie.MovieMetadata.Value.OriginalTitle    // Original title
    };

    // Add translations for wanted languages
    foreach (var translation in translations.Where(a => wantedLanguages.Contains(a.Language)
    {
        queryTranslations.Add(translation.Title);
    }

    spec.SceneTitles = queryTranslations
        .Where(t => t.IsNotNullOrWhiteSpace())
        .Distinct(StringComparer.InvariantCultureIgnoreCase)
        .ToList();
}
```

[!IMPORTANT] **Note**: The search uses **translations** based on quality profile language settings, but **alternative titles are NOT di-**

rectly included in indexer searches. Alternative titles are used for **matching parsed releases** back to movies.

---

4. Title Matching Logic

File: MovieService.cs

When Radarr needs to match a parsed release title to a movie, it uses a **cascading search strategy**:

```
public Movie FindByTitle(List<string> titles, int? year, List<string> otherTitles, List<Movi
{
    var cleanTitles = titles.Select(t => t.CleanMovieTitle().ToLowerInvariant());

    // 1. Try main title and original title
    var result = candidates
        .Where(x => cleanTitles.Contains(x.MovieMetadata.Value.CleanTitle) ||
                    cleanTitles.Contains(x.MovieMetadata.Value.CleanOriginalTitle))
        .AllWithYear(year)
        .ToList();

    // 2. Try other title variations (Roman numerals)
    if (result == null || result.Count == 0)
    {
        result = candidates
            .Where(movie => otherTitles.Contains(movie.MovieMetadata.Value.CleanTitle))
            .AllWithYear(year)
            .ToList();
    }

    // 3. Try alternative titles
    if (result == null || result.Count == 0)
    {
        result = candidates
            .Where(m => m.MovieMetadata.Value.AlternativeTitles.Any(t =>
                cleanTitles.Contains(t.CleanTitle) ||
                otherTitles.Contains(t.CleanTitle)))
            .AllWithYear(year)
            .ToList();
    }

    // 4. Try translations
    if (result == null || result.Count == 0)
    {
        result = candidates
```

```
            .Where(m => m.MovieMetadata.Value.Translations.Any(t =>
                cleanTitles.Contains(t.CleanTitle) ||
                otherTitles.Contains(t.CleanTitle)))
            .AllWithYear(year)
            .ToList();
    }
}
```

Search Priority: 1. Main title & original title 2. Roman numeral variations 3. **Alternative titles** (from TMDb, mappings, user, or indexer) 4. Translations

---

5. Title Candidate Generation

File: MovieService.cs

Before matching, Radarr generates title variations including Roman numeral conversions:

```
public List<Movie> FindByTitleCandidates(List<string> titles, out List<string> otherTitles)
{
    var lookupTitles = new List<string>();
    otherTitles = new List<string>();

    foreach (var title in titles)
    {
        var cleanTitle = title.CleanMovieTitle().ToLowerInvariant();
        var romanTitle = cleanTitle;
        var arabicTitle = cleanTitle;

        // Convert between Roman and Arabic numerals
        foreach (var arabicRomanNumeral in RomanNumeralParser.GetArabicRomanNumeralsMapping(
        {
            var arabicNumber = arabicRomanNumeral.ArabicNumeralAsString;
            var romanNumber = arabicRomanNumeral.RomanNumeral;

            romanTitle = romanTitle.Replace(arabicNumber, romanNumber);
            arabicTitle = arabicTitle.Replace(romanNumber, arabicNumber);
        }

        otherTitles.AddRange(new List<string> { arabicTitle, romanTitle });
        lookupTitles.AddRange(new List<string> { cleanTitle, arabicTitle, romanTitle });
    }

    return _movieRepository.FindByTitles(lookupTitles);
}
```

Example: “Rocky 2” generates: - `rocky2` (clean) - `rockyii` (Roman) - `rocky2` (Arabic)

---

6. Database Search Implementation

File: MovieRepository.cs

The repository performs three separate queries for efficiency:

```
public List<Movie> FindByTitles(List<string> titles)
{
    var distinct = titles.Distinct().ToList();
    var results = new List<Movie>();

    results.AddRange(FindByMovieTitles(distinct));    // Main/original titles
    results.AddRange(FindByAltTitles(distinct));      // Alternative titles
    results.AddRange(FindByTransTitles(distinct));    // Translations

    return results.DistinctBy(x => x.Id).ToList();
}
```

Alternative Title Query

```
private List<Movie> FindByAltTitles(List<string> titles)
{
    var builder = new SqlBuilder(_database.DatabaseType)
        .Join<AlternativeTitle, MovieMetadata>((t, mm) => t.MovieMetadataId == mm.Id)
        .Join<MovieMetadata, Movie>((mm, m) => mm.Id == m.MovieMetadataId)
        .Join<Movie, QualityProfile>((m, p) => m.QualityProfileId == p.Id)
        .LeftJoin<Movie, MovieFile>((m, f) => m.Id == f.MovieId)
        .Where<AlternativeTitle>(x => titles.Contains(x.CleanTitle));

    // Execute query and return movies
}
```

> [!NOTE] Separate queries are used instead of a combined query because SQLite’s query planner performs poorly with complex joins, resulting in table scans.

Complete Search Flow

```
graph TD
    A[User Initiates Search] --> B[ReleaseSearchService.MovieSearch]
    B --> C[Build Search Criteria]
    C --> D[Collect Titles]
```

```
D --> E[Main Title]
D --> F[Original Title]
D --> G[Translations for Wanted Languages]

E --> H[Send to Indexers]
F --> H
G --> H

H --> I[Indexers Return Releases]
I --> J[Parse Release Titles]
J --> K[Match to Movies]

K --> L[Try Main/Original Title]
L --> M{Match?}
M -->|Yes| N[Return Movie]
M -->|No| O[Try Roman Numeral Variations]
O --> P{Match?}
P -->|Yes| N
P -->|No| Q[Try Alternative Titles]
Q --> R{Match?}
R -->|Yes| N
R -->|No| S[Try Translations]
S --> T{Match?}
T -->|Yes| N
T -->|No| U[No Match]
```

Key Insights

Alternative Titles in Action

1. **Storage**: Alternative titles are stored in the `AlternativeTitles` table with normalized `CleanTitle` values
2. **Sources**: Can come from TMDb, external mappings, user input, or indexer responses
3. **Indexer Searches**: Alternative titles are **NOT** sent to indexers during searches
4. **Matching**: Alternative titles are used when **matching parsed release names** back to movies
5. **Priority**: Checked after main/original titles and Roman numeral variations, but before translations

Why This Design?

- **Efficiency**: Sending too many title variations to indexers would be inefficient

- **Relevance**: Indexers work best with primary titles and translations
- **Flexibility**: Alternative titles catch edge cases when parsing release names
- **Language Support**: Quality profile language settings determine which translations are searched

Example Scenario

Movie: "The Matrix" (1999) - **Main Title**: "The Matrix" - **Original Title**: "The Matrix" - **Alternative Titles**: "Matrix", " " (Japanese) - **Translations**: "La Matrice" (French), "Die Matrix" (German)

Search Process: 1. Indexer receives: "The Matrix", "La Matrice" (if French is in quality profile) 2. Indexer returns: "Matrix.1999.1080p.BluRay.x264" 3. Parser extracts: "Matrix" 4. Matching tries: - Main title "The Matrix" → No match (different clean title) - Alternative title "Matrix" → **Match!**

Relevant Files

- AlternativeTitle.cs - Model definition
- AlternativeTitleService.cs - Management logic
- ReleaseSearchService.cs - Search execution
- MovieService.cs - Title matching
- MovieRepository.cs - Database queries
- SearchCriteriaBase.cs - Search criteria model